

8/5/26, 11:20 AM

Student Submissions — Application Based Questions

SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

Student 25 **Perarasan V** 192521404RD

2 / 2 submitted

Q1. Student Grade Book Application

CODE

```
def calculate_grade(average):
    if average ≥ 90:
        return "O"
    elif average ≥ 80:
        return "A+"
    elif average ≥ 70:
        return "A"
    elif average ≥ 60:
        return "B+"
    elif average ≥ 50:
        return "B"
    else:
        return "F"

# (a) Dictionary storing student name as key and list of 5 subject marks as value
students = {
    "Arun": [85, 90, 78, 92, 88],
    "Bala": [45, 60, 55, 70, 40],
    "Chitra": [95, 92, 89, 96, 91],
    "Divya": [70, 65, 72, 68, 75],
    "Elango": [55, 58, 62, 50, 60]
}

# (b) Computing total, average, and grade using dictionary comprehension
results = {
    name: {
        "marks": marks,
        "total": sum(marks),
        "average": round(sum(marks) / len(marks), 2),
        "grade": calculate_grade(sum(marks) / len(marks))
    }
    for name, marks in students.items()
}
```

```

    }
    for name, marks in students.items()
}

# (c) Finding the class topper (highest average)
topper = max(results, key=lambda name: results[name]["average"])

# Finding subject with highest overall mean
subject_names = ["Subject1", "Subject2", "Subject3", "Subject4", "Subject5"]
subject_totals = [0] * 5

for marks in students.values():
    for i in range(5):
        subject_totals[i] += marks[i]

subject_means = [round(total / len(students), 2) for total in subject_totals]
highest_subject_index = subject_means.index(max(subject_means))
highest_subject = subject_names[highest_subject_index]

# (d) Displaying formatted result table sorted by average in descending order
sorted_students = sorted(results.items(), key=lambda item: item[1]["average"], reverse=True)

print("=" * 70)
print(f"{'Name':<10}{'Marks':<25}{'Total':<8}{'Average':<10}{'Grade':<6}")
print("=" * 70)

for name, data in sorted_students:
    marks_str = str(data["marks"])
    print(f"{'name':<10}{'marks_str':<25}{'data['total']':<8}{'data['average']':<10}{'data['grade']':<6}")

print("=" * 70)

print(f"\nClass Topper: {topper} with Average = {results[topper]['average']} and Grade = {results[topper]['grade']}")

```

```
print("\nSubject-wise Mean Marks:")
for i in range(5):
    print(f"{subject_names[i]}: {subject_means[i]}")
```

simatslab.saveetha.com/practice_app/classactivity/ajax/download_submissions.php

43/60

8/5/26, 11:20 AM

Student Submissions — Application Based Questions

SIMATS Engineering • CSE • CSA0804 — Python Programming • 05-08-2026 • Category: Application Based Questions • Student Submissions Report

```
print(f"\nSubject with Highest Overall Mean: {highest_subject} ({max(subject_means)})")
```

OUTPUT

(no output)

Q2. Debugging and Rewriting

CODE

```
accounts = {}

def create_account(acc_no, name, balance):
    accounts[acc_no] = {'name': name, 'balance': balance, 'txns': []}

def deposit(acc_no, amount):
    if acc_no in accounts:
        accounts[acc_no]['balance'] += amount
        accounts[acc_no]['txns'].append(('Deposit', amount))

def withdraw(acc_no, amount):
    if acc_no in accounts:
        if accounts[acc_no]['balance'] > amount:
            accounts[acc_no]['balance'] -= amount
```

```
        accounts[acc_no]['txns'].append(('Withdraw', amount))
    else:
        print('Insufficient balance')

def mini_statement(acc_no):
    if acc_no in accounts:
        for t in accounts[acc_no]['txns'][-5:]:
            print(t)

create_account(1001, 'Ravi', 10000)
deposit(1001, 5000)
withdraw(1001, 20000)
withdraw(1001, 3000)
mini_statement(1001)
print('Balance:', accounts[1001]['balance'])
```

OUTPUT

(no output)